

PlugPlan SG AI Technology Stack Critique

SC2006 Team SCE3-04 | Lab 2 | 11 September 2026

1 Material supplied and method

The AI received the Lab 1 system description with the agreed FR-24 and UC-07 moderation refinement, and the Lab 2 conceptual entity class diagram. The material covers Driver and Moderator roles, vehicle profiles, immutable Plan Versions, provider snapshots, recommendations and time-limited issue moderation. Full reproductions are retained in the repository rather than repeated here.

The response below was produced by an independent GPT-5.6-luna subagent configured at xhigh reasoning effort, using local file access.

2 Exact prompt

You are an independent AI reviewer for SC2006 Lab 2. Read D:/Downloads/SC2006-SCE3-04/lab2/source/system_description.md and inspect all three supplied conceptual class diagram images using view_image: D:/Downloads/SC2006-SCE3-04/lab2/diagrams/entity-01-ownership.drawio.png D:/Downloads/SC2006-SCE3-04/lab2/diagrams/entity-02-calculation.drawio.png D:/Downloads/SC2006-SCE3-04/lab2/diagrams/entity-03-moderation.drawio.png

Based on these design materials, recommend a technology stack appropriate for an entry-level software engineering team. Explain how the proposed technologies support the depicted entities, immutable Plan Versions, provider adapters, role-restricted moderation and the rank-then-filter faulty-station rule. Identify avoidable complexity and a simpler alternative where appropriate. The team's existing technology experience has not been supplied; do not invent it. Treat this as a provisional discussion for a course project, not a binding design or a claim of implementation. Preserve the recommendation-only scope and manual refresh.

Write a concise original response in English, approximately 300–400 words, including at least one technology that is not a programming language. Verify current technology capability claims using primary official documentation and include 2–4 source URLs. Use plain text paragraphs or numbered items, without Markdown headings or tables. Save your complete response verbatim to D:/Downloads/SC2006-SCE3-04/lab2/ai-evidence/response.md and return the identical text as your final answer. Do not edit the supplied inputs or write the student's selected-technology opinion. This is read-only review and lightweight local document writing; do not use a compute cluster.

3 Original AI response

1. Recommend a Python FastAPI modular monolith, PostgreSQL, and a thin browser UI using server-rendered HTML or a small JavaScript client. FastAPI's type-based validation, automatic OpenAPI documentation, and dependency system suit a small course API; security dependencies can express authentication and role or scope checks: <https://fastapi.tiangolo.com/features/> and <https://fastapi.tiangolo.com/advanced/security/oauth2-scopes/>. This is a discussion recommendation; the team's existing experience has not been supplied.

2. Map the conceptual entities to normalized PostgreSQL tables: User, EVProfile, ChargingPlan, PlanVersion, OfficialSnapshot, RouteEstimate, Recommendation, Connector, ChargingStation, IssueReport, and ModerationDecision. Foreign keys connect ownership and composition; primary-key, UNIQUE, NOT NULL, and CHECK constraints protect identifiers, cardinality, and ranges. PostgreSQL documents these integrity constraints here: <https://www.postgresql.org/docs/current/ddl-constraints.html>. Enforce "only a Driver owns a profile or plan" in the service authorization check too; a foreign key alone does not understand the role value.

3. Treat PlanVersion as append-only data. Reconfiguration or manual refresh inserts a version and its RouteEstimate/Recommendation children, records the exact OfficialSnapshot and copied assumptions, then updates the plan's current-selection pointer in one PostgreSQL transaction. Transactions provide all-or-nothing visibility: <https://www.postgresql.org/docs/current/tutorial-transactions.html>. Expose no update or delete operations for historical versions; keep selection mutable at ChargingPlan level. A small repository layer and deterministic tests are enough; event sourcing adds avoidable complexity.

4. Define small MapProvider and StationProvider interfaces, with OneMap, LTA DataMall, cached, and Demo Fixture adapters returning one canonical shape. Persist source, observed/retrieved timestamps, and data state so ranking is provider-independent. The recommendation service calculates feasibility and ranks all candidates first, then reads active verified, unexpired faults, marks affected recommendations FILTERED_FAULT, and fills the shortlist from next-ranked unmarked candidates (up to three). Pending reports never enter this exclusion query; ModerationDecision records actor, reason, action, timestamp, and resulting status. Put moderator endpoints behind a server-side role dependency, and use a maintained password-hashing library plus TLS at deployment.

5. Keep manual refresh as an explicit endpoint. Avoid microservices, GraphQL, a message broker, Redis, and continuous monitoring. For a single-process local demo, SQLite with the same schema and server-rendered pages is simpler; PostgreSQL is preferable for shared testing and relational constraints.

4 Opinion on one proposed technology

PostgreSQL is a reasonable provisional choice for PlugPlan SG because related plans, immutable calculation records and moderation decisions benefit from relational constraints and atomic transactions. Its setup and administration are an added cost for a short course project, so SQLite remains a simpler option for a single-process demonstration until shared testing justifies PostgreSQL. We would reject the AI's blanket prohibition on deleting historical versions: immutability should prevent in-place edits, while deleting a saved plan must still make its data inaccessible through the application within 60 seconds under NFR-07.